Knowhere to Grow

A space to learn.
A place to grow.

A secure learning management system for students and instructors.

[Institution Name] · [Program / Course / Section]
[Student Names] · 15 October 2026



Learning gets scattered.
Progress gets harder to follow.

01

Resources in many places

Students spend time finding the right files, links, and instructions.

02

Unclear submission status

Messages and shared folders make deadlines and submitted work difficult to track.

03

Feedback feels disconnected

Grades and comments need to connect clearly to the work students submitted.

Project context describes a design problem; no institutional survey or measured impact is claimed.

One shared learning space. Two purposeful experiences.

STUDENT · WEB + FLUTTER

Learn. Submit. Improve.

Join courses, access resources, follow deadlines, submit work, and act on feedback.

INSTRUCTOR · WEB

Organize. Guide. Assess.

Manage courses and resources, create assignments, review submissions, and give written feedback.

Connected clients.
A single source of academic records.

React web

React Router · Hooks · Axios · Bootstrap

Student + instructor

Flutter mobile

Material UI · HTTP · Secure storage

Student only

→

Express + Node.js

REST API

JWT authentication

Validation and authorization

→

MongoDB

Academic records

Revocable sessions

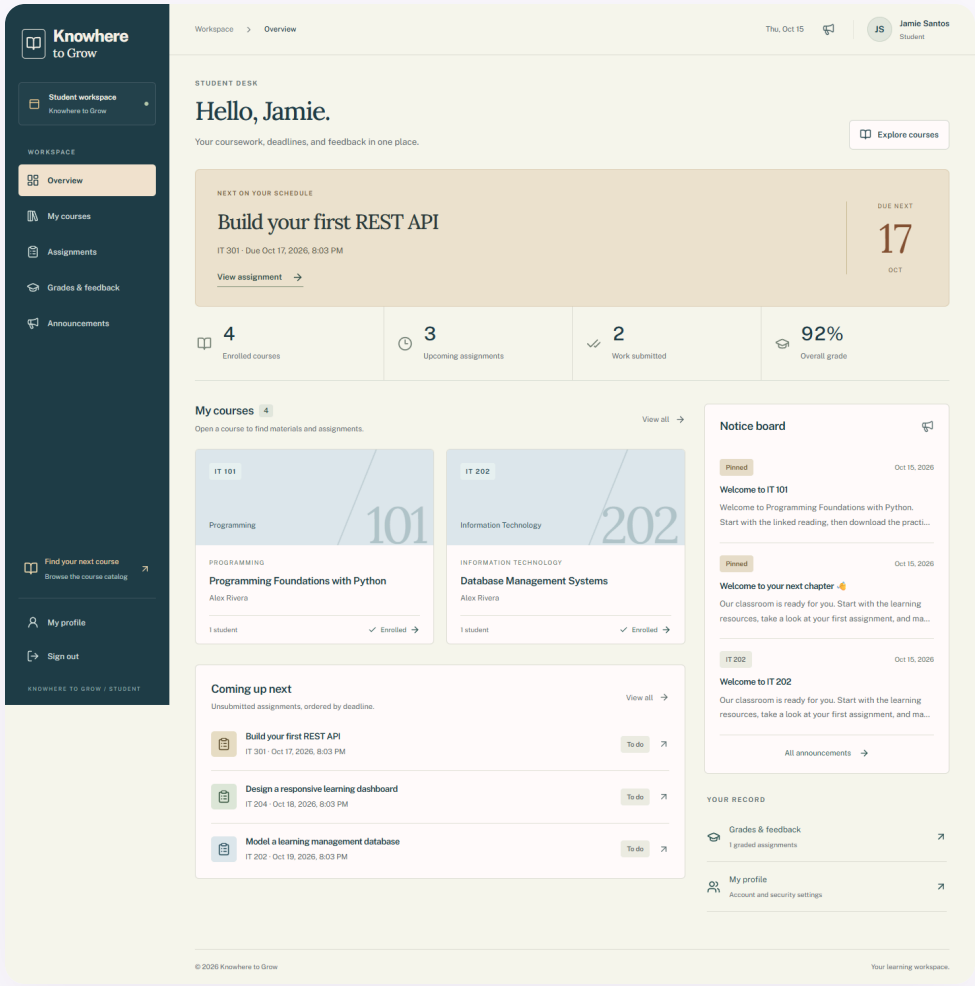
GridFS attachments

Neither client receives database credentials or accesses MongoDB directly.

Know what's next.

Courses, upcoming assignments, submitted work, grades, and announcements in one responsive workspace.

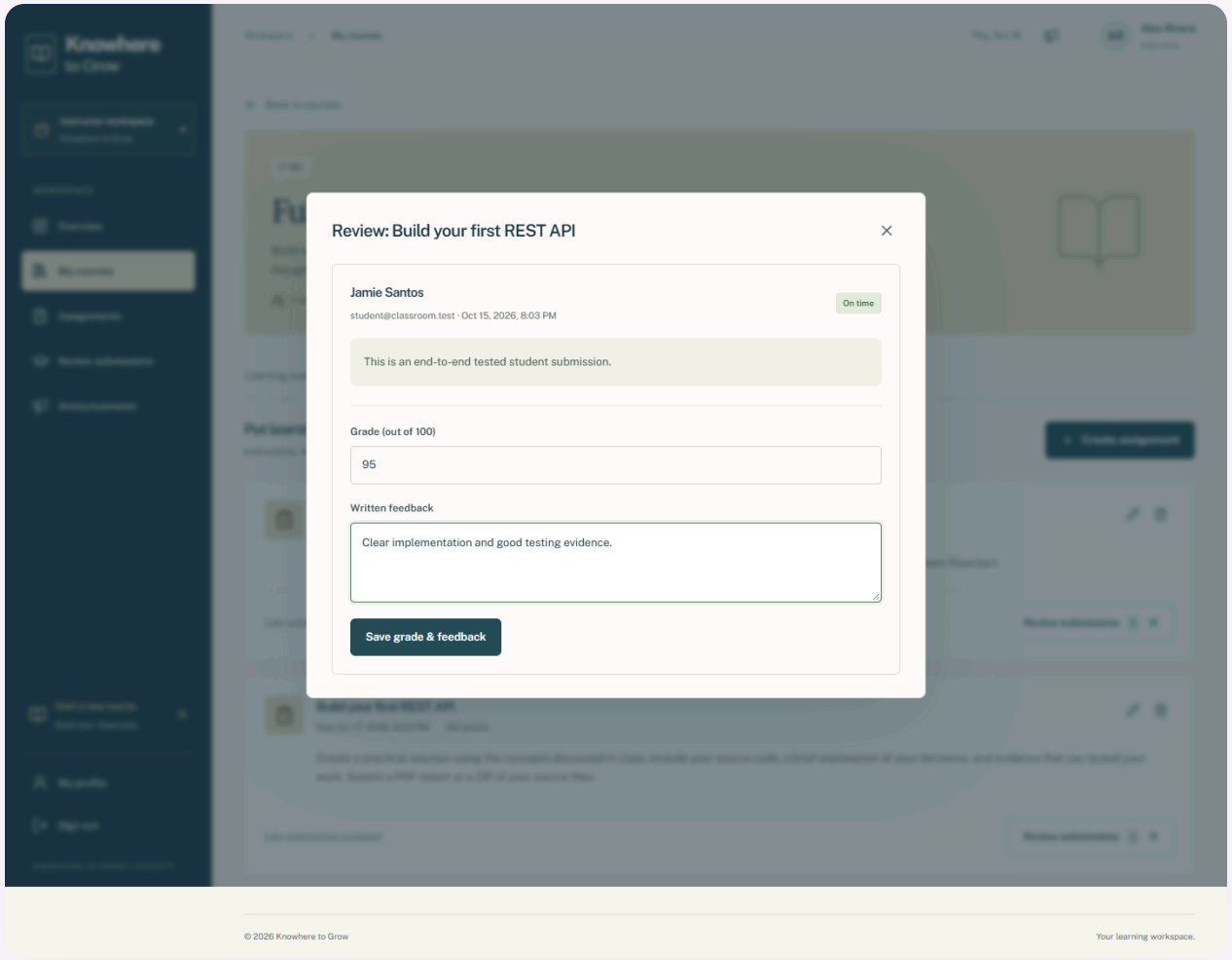
- Search and pagination
- Phone navigation
- Profile and password management



Turn effort into direction.

Create and publish courses. Share resources, manage assignments, and provide useful feedback.

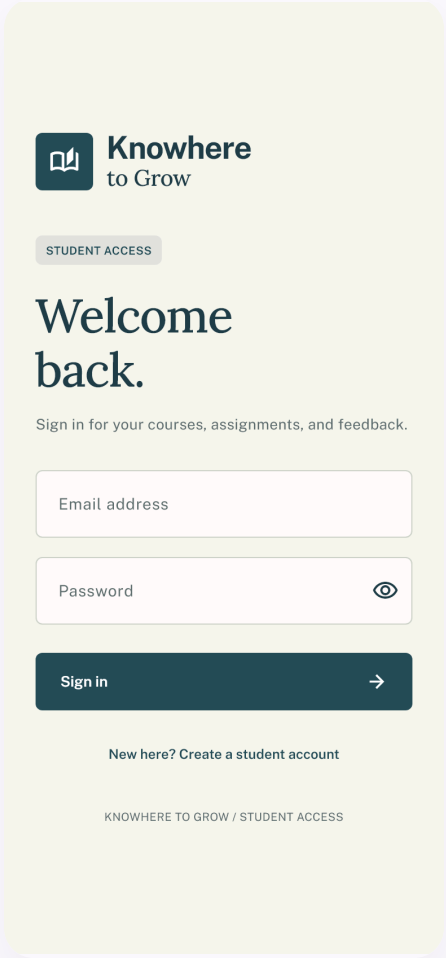
- Own-course management
- Student submissions and statistics
- Grades and written comments

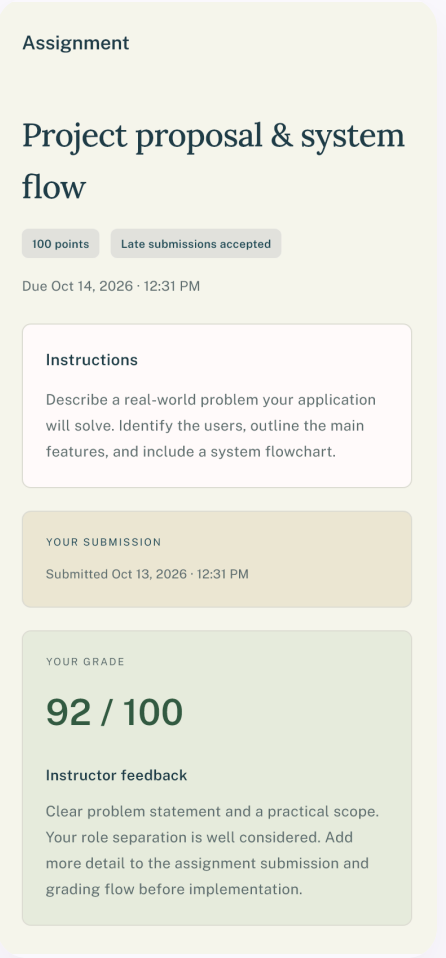


Your classroom, closer at hand.

Student login and registration, enrolled courses, resources, assignments, grades, updates, and profile management.

Both clients share the same API and database. Instructor mobile authentication is rejected by the server.





Actual Flutter widget renders. Android SDK/device execution remains to be verified.

A clear path from task to growth.

1

Create

Instructor sets instructions, deadline, points, and late policy.

2

Submit

Enrolled student submits text and/or an attachment.

3

Review

Course owner assesses work and saves a valid grade plus feedback.

4

Improve

Student views the same result on web or Flutter.

The API enforces deadlines, 3 MB attachment limits, enrollment, and graded-submission locking.

Permissions live on the server.

Identity

Bcrypt password hashes.

Signed JWT with expiry.

Persisted, revocable sessions.

Authorization

Role middleware.

Course ownership checks.

Enrollment and submission-owner checks.

Input and transport

Zod validation.

Origin and rate controls.

Protected downloads.

HTTPS required in production.

Web tokens: HTTP-only cookies. Mobile tokens: platform secure storage. No password hashes in API responses.

Evidence behind the implementation.

30

API / startup tests

Workflows, limits, serverless runtime

5

Browser flows passed

Desktop and phone viewport

15

Flutter tests passed

API, authentication, rendering

0

Known npm vulnerabilities

Audited installed lockfiles

React build and lint pass. Flutter analysis is clean. A live Atlas-backed student API session was verified.

No physical-device, emulator, APK, or iOS build success is claimed.

Focused academic scope. Clear next steps.

Implemented

Course management, learning materials, assignments, submissions, grades, announcements, profiles, shared API, and student-only Flutter access.

Outside scope / pending

No payments, live video, quizzes, attendance, push alerts, offline submissions, or password-reset email. Android SDK setup and device smoke testing remain; iOS needs macOS/Xcode.

Production also needs HTTPS, managed secrets, backups, monitoring, and release signing.

Show the same learning journey from both sides.

Instructor web

Create course → upload material → create assignment

↓

Student web / Flutter

Join course → read resource → submit work

↓

Feedback loop

Instructor grades → student reads grade and feedback

Use demonstration accounts. Keep database passwords, invitation codes, and environment files off screen.